

CUDAGraph#

<https://pytorch.org/docs/stable/generated/torch.cuda.CUDAGraph.html>

Description:

Wrapper around a CUDA graph. `keep_graph` (bool, optional) – If `keep_graph=False`, the `cudaGraphExec_t` will be instantiated on GPU at the end of `capture_end` and the underlying `cudaGraph_t` will be destroyed. Users who want to query or otherwise modify the underlying `cudaGraph_t` before instantiation can set `keep_graph=True` and access it via `raw_cuda_graph` after `capture_end`. Note that the `cudaGraphExec_t` will not be instantiated at the end of `capture_end` in this case. Instead, it will be instantiated via an explicit `called` to `instantiate` or automatically on the first call to `replay` if `instantiate` was not already called. Calling `instantiate` manually before `replay` is recommended to prevent increased latency on the first call to `replay`. It is allowed to modify the raw `cudaGraph_t` after first calling `instantiate`, but the user must call `instantiate` again manually to make sure the instantiated graph has these changes. Pytorch has no means of tracking these changes. This API is in beta and may change in future releases.

Function Signature

```
class torch.cuda.CUDAGraph(keep_graph=False)[source]#
```

Parameters

Return type

```
capture_begin(pool=None, capture_error_mode='global')  
[source]#
```

Parameters

Returns:

int

Notes & Warnings

WARNING

Warning This API is in beta and may change in future releases.

Detailed Documentation

Documentation

Wrapper around a CUDA graph.

`keep_graph` (bool, optional) – If `keep_graph=False`, the `cudaGraphExec_t` will be instantiated on GPU at the end of `capture_end` and the underlying `cudaGraph_t` will be destroyed. Users who want to query or otherwise modify the underlying `cudaGraph_t` before instantiation can set `keep_graph=True` and access it via `raw_cuda_graph` after `capture_end`. Note that the `cudaGraphExec_t` will not be instantiated at the end of `capture_end` in this case. Instead, it will be instantiated via an explicit call to `instantiate` or automatically on the first call to `replay` if `instantiate` was not already called. Calling `instantiate` manually before `replay` is recommended to prevent increased latency on the first call to `replay`. It is allowed to modify the raw `cudaGraph_t` after first calling `instantiate`, but the user must call `instantiate` again manually to make sure the instantiated graph has these changes. Pytorch has no means of tracking these changes.

This API is in beta and may change in future releases.

Begin capturing CUDA work on the current stream.

Typically, you shouldn't call `capture_begin` yourself. Use `graph` or `make_graphed_callables()`, which call `capture_begin` internally.

`pool` (optional) – Token (returned by `graph_pool_handle()` or `other_Graph_instance.pool()`) that hints this graph may share memory with the indicated pool. See Graph memory management.

`capture_error_mode` (str, optional) – specifies the `cudaStreamCaptureMode` for the graph capture stream. Can be “global”, “thread_local” or “relaxed”. During cuda graph capture, some actions, such as `cudaMalloc`, may be unsafe. “global” will error on actions in other threads, “thread_local” will only error for actions in the current thread, and “relaxed” will not error on these actions. Do NOT change this setting unless you're familiar with `cudaStreamCaptureMode`

End CUDA graph capture on the current stream.

After `capture_end`, `replay` may be called on this instance.

Typically, you shouldn't call `capture_end` yourself. Use `graph` or `make_graphed_callables()`, which call `capture_end` internally.

`debug_path` (required) – Path to dump the graph to.

Calls a debugging function to dump the graph if the debugging is enabled via `CUDAGraph.enable_debug_mode()`

Enable debugging mode for `CUDAGraph.debug_dump`.

Instantiate the CUDA graph. Will be called by `capture_end` if `keep_graph=False`, or by `replay` if `keep_graph=True` and `instantiate` has not already been explicitly called. Does not destroy the `cudaGraph_t` returned by `raw_cuda_graph`.

Return an opaque token representing the id of this graph's memory pool.

This id can optionally be passed to another graph's `capture_begin`, which hints the other graph may share the same memory pool.

Returns the underlying `cudaGraph_t`. `keep_graph` must be `True`.

See the following for APIs for how to manipulate this object: [Graph Managment](#) and [cuda-python Graph Management bindings](#)

Returns the underlying `cudaGraphExec_t`. `instantiate` must have been called if `keep_graph` is `True`, or `capture_end` must have been called if `keep_graph` is `False`. If you call `instantiate()` after `raw_cuda_graph_exec()`, the previously returned `cudaGraphExec_t` will be destroyed. It is your responsibility not to use this object after destruction.

See the following for APIs for how to manipulate this object: [Graph Execution](#) and [cuda-python Graph Execution bindings](#)

Replay the CUDA work captured by this graph.

Delete the graph currently held by this instance.